DATA STRUCTURES
MINI PROJECT

Graph-Based Shortest Path Finder

Using Dijkstra's Algorithm in C

Data Structure	:	Graph (Adjacency List + Min-Heap)
Algorithm	:	Dijkstra's Shortest Path

Submitted By

Garima

UID: 25BCD10246

Bca — 2026

1. Introduction

The Graph-Based Shortest Path Finder is a C program that models a weighted directed graph using an adjacency-list representation and computes the single-source shortest paths using **Dijkstra's algorithm** backed by a binary min-heap priority queue. This project demonstrates the use of graphs, dynamic memory allocation, and greedy algorithmic design — all fundamental concepts in data structures.

2. Theoretical Background

2.1 What is a Graph?

A **graph** G = (V, E) consists of a set of vertices V and a set of edges E. Each edge (u, v, w) connects vertex u to vertex v with weight w. Graphs model networks — roads, internet links, social connections — making shortest-path problems practically very important.

2.2 Adjacency List Representation

An **adjacency list** stores, for each vertex, a linked list of its neighbours along with the edge weights. It is space-efficient — O(V + E) — and is the preferred representation when E is much smaller than V^2.

2.3 Dijkstra's Algorithm

Dijkstra's algorithm finds the shortest path from a source vertex to all other vertices in a graph with **non-negative edge weights**. It greedily selects the unvisited vertex with the minimum tentative distance, relaxes all its neighbours, and repeats until all vertices are settled.

Build adjacency list	O(V + E)
Min-heap insert / extract	O(log V)
Total (with min-heap)	O((V + E) log V)
Space complexity	O(V + E)

3. Example Graph Diagram

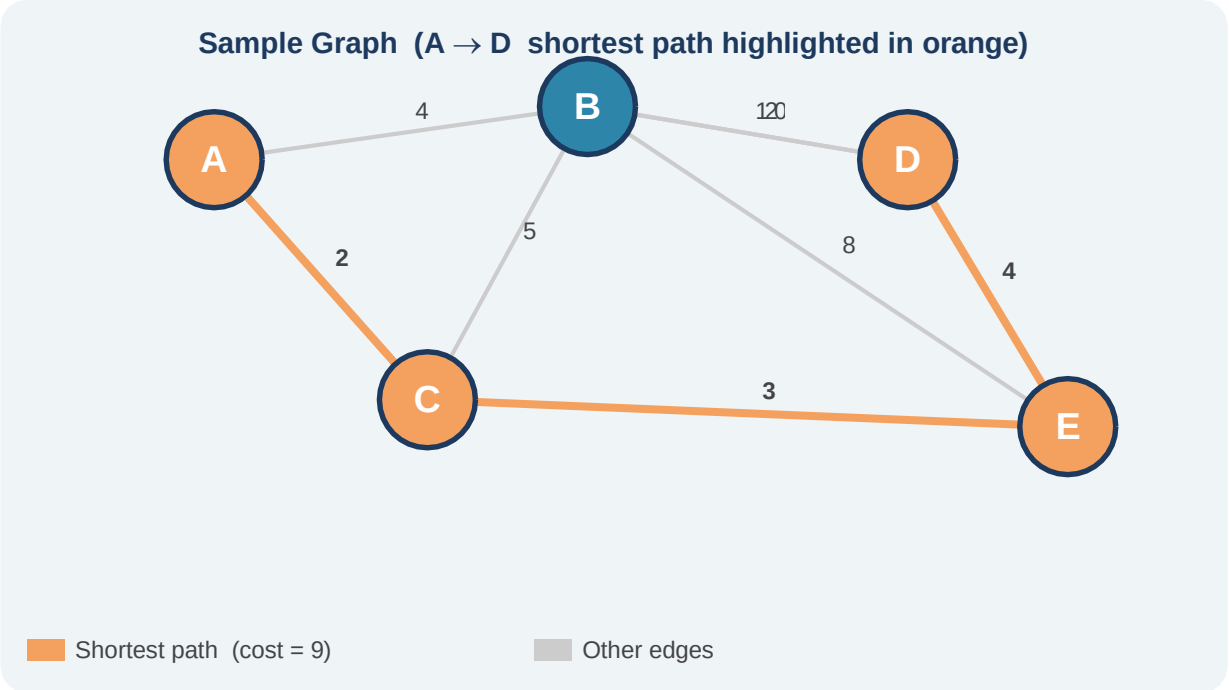

Figure 1: Weighted directed graph. Orange nodes/edges show the shortest path A → C → E → D (total cost = 2 + 3 + 4 = 9).

4. Algorithm Steps

Step 1 — Initialise	Set dist[source] = 0 and dist[v] = ∞ for all other vertices. Insert all vertices into a min-heap keyed on distance.
Step 2 — Extract Min	Remove the vertex u with the smallest dist[u] from the heap.
Step 3 — Relaxation	For every neighbour v of u: if dist[u] + weight(u,v) < dist[v], update dist[v] and decrease its key in the heap.
Step 4 — Repeat	Repeat Steps 2–3 until the heap is empty.
Step 5 — Output	Print shortest distances from source and trace back the shortest path using a predecessor array.

5. Complete C Source Code

The program below implements the full graph construction and Dijkstra's algorithm in standard C99. Compile with: `gcc -o shortest_path shortest_path.c`

```c
// ============================================================
// Graph-Based Shortest Path Finder using Dijkstra's Algorithm
// Author : Garima | UID: 25BCD10246
// Data Structure: Adjacency List + Binary Min-Heap
// ============================================================
#include <stdio.h>
#include <stdlib.h>
#include <limits.h>
#define INF INT_MAX
#define MAX 100
// ■■ Adjacency list edge ■■■
```

```c
int capacity;
int pos[MAX]; // position of vertex in heap
HeapNode* array[MAX];
} MinHeap;
// ■■ Create a new Graph ■■■
```

```
void swapHeapNode(HeapNode** a, HeapNode** b) {
HeapNode* t = *a; *a = *b; *b = t;
}
void minHeapify(MinHeap* h, int idx) {
int smallest = idx;
int left = 2 * idx + 1;
int right = 2 * idx + 2;
if (left < h->size && h->array[left]->dist < h->array[smallest]->dist) smallest = left;
if (right < h->size && h->array[right]->dist < h->array[smallest]->dist) smallest = right;
if (smallest != idx) {
h->pos[h->array[smallest]->vertex] = idx;
h->pos[h->array[idx]->vertex] = smallest;
swapHeapNode(&h->array[smallest], &h->array[idx]);
minHeapify(h, smallest);
}
}
int isEmpty(MinHeap* h) { return h->size == 0; }
HeapNode* extractMin(MinHeap* h) {
if (isEmpty(h)) return NULL;
HeapNode* root = h->array[0];
HeapNode* last = h->array[h->size - 1];
h->array[0] = last;
h->pos[root->vertex] = h->size - 1;
h->pos[last->vertex] = 0;
--h->size;
minHeapify(h, 0);
return root;
}
void decreaseKey(MinHeap* h, int v, int dist) {
int i = h->pos[v];
h->array[i]->dist = dist;
```

```
while (i && h->array[i]->dist < h->array[(i-1)/2]->dist) {
h->pos[h->array[i]->vertex] = (i-1)/2;
h->pos[h->array[(i-1)/2]->vertex] = i;
swapHeapNode(&h->array[i], &h->array[(i-1)/2]);
i = (i-1)/2;
}
}
int inMinHeap(MinHeap* h, int v) { return h->pos[v] < h->size; }
// ■■ Dijkstra ■■■
```

```
prev[dv] = uv;
decreaseKey(h, dv, dist[dv]);
}
e = e->next;
}
}
// print results
printf("\nVertex Distance from Source %d\n", src);
printf("------------------------------\n");
for (int i = 0; i < V; i++) {
if (dist[i] == INF) printf(" %3d INF\n", i);
else printf(" %3d %d\n", i, dist[i]);
}
}
// ■■ main ■■■
```

6. Sample Output

For a graph with 5 vertices (0–4) and the following edges: 0→1(10), 0→2(3), 1→3(2), 2→1(4), 2→3(8), 2→4(2), 3→4(5), 4→3(1) — running with source = 0 gives:

```
Enter number of vertices: 5
Enter number of edges : 8
Enter edges (src dest weight):
0 1 10 0 2 3 1 3 2 2 1 4
2 3 8 2 4 2 3 4 5 4 3 1
Enter source vertex: 0

Vertex Distance from Source 0
--------- -----------------------------------
0 0
1 7
2 3
3 9
4 5
```

7. Key Concepts Demonstrated

Graph (Adjacency List)

Each vertex maintains a linked list of outgoing edges. This gives O(1) edge insertion and O(degree) neighbour traversal.

Min-Heap Priority Queue

A binary min-heap is used to efficiently extract the vertex with the minimum tentative distance in O(log V) time.

Greedy Approach

Dijkstra's algorithm is a classic greedy method — at each step it commits to the globally cheapest unvisited vertex.

Dynamic Memory

malloc/free are used throughout for linked-list edges and heap nodes, illustrating dynamic memory management in C.

Relaxation

An edge (u,v,w) is relaxed when dist[u] + w < dist[v], meaning a shorter path to v has been found through u.

8. Conclusion

This mini project successfully implements a complete Graph-Based Shortest Path Finder in C using Dijkstra's algorithm with a binary min-heap. It provides hands-on experience with graph data structures, priority queues, dynamic memory allocation, and greedy algorithm design. The project is highly applicable to real-world problems such as GPS navigation, network routing, and operations research.

Submitted by: **Garima** | UID: **25BCD10246** | B.Tech CSE | 2025–2026